

Introduction

The provided agent instructions describe an ambitious, highly-structured autonomous agent framework. The system is designed to **develop an entire web application or solve complex tasks end-to-end** in a professional, production-ready manner. It defines a **phase-based lifecycle**, specialized modules (and sub-agents), a knowledge graph for context, strict documentation practices, and extensive guardrails for safety and quality. Below, we **analyze and map out the entire process in detail**, including a hierarchical “tree-of-thought” diagram of the process, a step-by-step mapping of all phases down to atomic actions, and a critical evaluation of the design with recommendations for improvement.

Comprehensive Features and Process Elements Overview

The agent framework encompasses numerous features, loops, and behavior management mechanisms. Key elements inferred from the instructions include:

- **Phase-Based Lifecycle:** Eight sequential phases structure the workflow: **Context Gathering, Analysis, Research & Synthesis, Brainstorm & Evaluation, Planning, Execution, Review & Reflection**, and **Delivery** ¹ ². Each phase has a clear goal, defined steps, loop conditions, and outputs. Phases depend on the completion of previous phases' tasks (e.g. Execution depends on Planning) ³, ensuring a logical progression of work. However, if new information arises (e.g. a change in requirements), the process can loop back to an earlier phase (especially Planning) to update the plan ⁴ ⁵.
- **Main Agent Loop (ReAct Pattern):** A continuous **Thought → Action → Observation → Iterate** loop controls execution ⁶. In each cycle, the agent:
 - **Analyzes events/state:** Reads the latest user input and internal state (event log, plan, tasks) to determine the current goals and active phase ⁷. If a new phase starts, a **PhaseChange** event is logged.
 - **Chooses a module (Thought):** Decides the next course of action and selects the appropriate **module or sub-agent** for the current phase (e.g. use `research_module` during the Research phase) ⁶ ⁸. A **ModuleSelection** event is logged with the chosen module/agent.
 - **Executes an action:** Performs **one tool or function call** through the selected module/sub-agent (e.g. run a web search, edit code, call an API) ⁹. Only one action is taken per iteration, following guardrails (no side-effects without confirmation). An **Action** event is logged describing the tool and purpose.
 - **Observes result:** Waits for the tool's result and processes the outcome. The result is summarized and recorded as an **Observation** event ⁹. Relevant data or sources are cited, and the knowledge base (memory or graph) is updated if needed.
 - **Iterates or transitions:** Checks if the current task/phase is complete. If not, the loop repeats (back to analysis step) with updated context. If the phase's goal is achieved or tasks done, the agent prepares to move to the next phase or to deliver results ¹⁰. On phase completion, it may log a

Delivery event and compile outputs for the user ¹¹. The agent continues looping until all project phases are finished or the user stops the process.

- **Specialized Modules and Sub-Agents:** The system is composed of modular functionalities, each encapsulated in **modules** (original design) or **sub-agents** (improved design). Major modules and their roles include ¹² ¹³ :

- **Context Module / Manager:** Gathers initial context, infers user goals, loads relevant files (e.g. `planning.md`, `todo.md`, prior research notes) and summarises the situation ¹⁴. It uses context-engineering strategies to include only the most relevant information in the prompt (avoiding context overflow) ¹⁴. This module ensures the agent starts with a **clear understanding of goals and constraints**.
- **Analysis Module / Agent:** Identifies key entities, concepts, requirements, and their relationships from the context. It builds a hierarchical **tree-of-thought (ToT) diagram** or graph to map these entities and relations ¹⁵. The map (often saved as `TOT.md`) captures the problem structure and is kept updated if new entities arise.
- **Research Module / Agent:** Performs in-depth research by querying authoritative sources (documentation, APIs, academic articles, etc.) ¹⁶. It gathers evidence and best practices related to the problem, cross-validates information (with a safety/compliance check on external info) ¹⁶, and synthesizes findings into a research report (saved as `RESEARCH.md`) with proper citations ¹⁷.
- **Brainstorm Module / Agent:** Generates ideas, potential solutions or prompt components, and evaluates them ¹⁸. It encourages divergent thinking to cover many approaches (including optional enhancements like multi-agent teamwork or different architectures). Each idea is assessed for feasibility/impact, given a score, and the top-ranked ideas are recorded in `BRAINSTORM.md` ¹⁹.
- **Planning Module / Task Agent:** Develops a **detailed project plan** from the chosen ideas and research insights ²⁰. It breaks the project into tasks, organizes them into phases, and records a step-by-step roadmap in `planning.md` and a checklist in `todo.md` ²¹. This plan includes dependencies, sequencing, and assignment of tasks to modules or sub-agents. The module also consults a “virtual expert panel” – simulating perspectives of experts in requirements, architecture, UX, etc. – to review and refine the plan ²². Plan changes are logged as **PlanUpdate** events.
- **Execution Module / Agent:** Carries out the tasks according to the plan, one by one ²³. It may invoke **domain-specific sub-agents** (like a frontend developer agent, backend agent, testing agent, etc.) to implement different parts of the solution ²⁴. The execution agent writes or modifies code, configures systems, and uses tools (e.g., running test suites, database migrations, etc.) as needed. **Test-driven development** is emphasized: write failing tests first, implement code to pass them, then run all tests (unit, integration, performance, etc.) ²⁵. After each action, it logs the outcome and marks tasks complete in `todo.md` ²⁶. If an error occurs, an **Error** event is logged and a fix is attempted ²⁷.
- **Review & Reflection Module / Agent:** Critically evaluates the outputs and process, simulating a panel of expert reviewers ²⁸. It reviews code quality, design, performance, security, etc., using static analysis tools and audits. It logs reflections or lessons as **KnowledgeCapture** events and may identify bugs or improvements, which become new tasks (e.g. logged in a bug list or fed back into Planning) ²⁹ ³⁰. This phase iterates until all critical issues are resolved ³¹.
- **Delivery Module / Agent:** Finalizes and delivers the results to the user ³². It verifies all checklist items are done and all artifacts are up to date ³³. Then it compiles deliverables: syncing files such as the research report, ToT diagram, brainstorming notes, final code or documents, etc., via

`computer.sync_file` ³⁴. The improved agentic prompt and process description (CLAUDE.md and CLAUDE_PROCESS.md) are also presented in full as part of the output ³⁵. After delivery, the agent enters a standby state awaiting further instructions ³⁶.

- **Cross-Cutting Modules and Safeguards:** Several **global behaviors** apply across all phases:
- **Memory Management & Knowledge Graph:** A **persistent memory** stores context and a **knowledge graph** models the project. Important data (plans, research summaries, decisions) are saved with keys (e.g. `plan-2025-08-13`) so they can be recalled later. The knowledge graph contains **entities** (Tasks, Phases, Modules, Files, Requirements, etc.) and **relations** (e.g. Task *depends_on* Requirement, Task *belongs_to* Phase, Component *uses* API) ³ ³⁷. This graph serves as a “system of context,” capturing the state and dependencies of the project for reasoning. The agent creates and updates graph nodes via tools like `context7.create_entities` and `create_relations`, and queries it (`search_nodes`, `open_nodes`) to avoid hallucination and track progress ³⁸ ³⁹. Memory is also used for “scratchpads” to store intermediate results or plans outside the immediate LLM context (for context window management).
- **Logging and Event Tracking:** Every significant action is logged to an `event-stream.md` with a timestamp and event type ⁴⁰ ⁷. Event types include:
 - **UserMessage:** recording user inputs,
 - **PhaseChange:** entering a new phase,
 - **ModuleSelection:** choosing a module/subagent for an action,
 - **Action:** executing a tool or function,
 - **Observation:** result of an action,
 - **PlanUpdate:** modifying the plan or tasks (e.g. adding or completing a task),
 - **KnowledgeCapture:** recording a piece of learned information, decision rationale, or best practice (often with a DataSource/citation if from research),
 - **Error:** noting any exception or issue encountered,
 - **Delivery:** final output delivery event. These logs ensure transparency and traceability of the agent’s decisions ⁴⁰. They also help with debugging and reviewing the agent’s reasoning retrospectively.
- **Guardrails (Safety & Compliance):** A guardrails module (or **safety-compliance agent**) runs throughout to enforce safe and correct behavior ⁴¹. It verifies external information by cross-checking sources (to prevent accepting false or malicious content) ¹⁶, ignores any unexpected instructions from web pages, and sanitizes inputs/outputs (e.g. escaping shell commands, preventing code injection). It ensures no sensitive data is leaked and that any action with side effects (like making a purchase, sending an email, deploying code) is confirmed by the user before execution ⁴² ⁴³. This module also enforces **compliance** with regulations (like GDPR, HIPAA if applicable) and proper secrets management (use env variables for API keys, etc.). If the agent encounters a potential policy violation or high-risk situation, it will stop or seek clarification.
- **Design & UX Standards:** A cross-cutting design module ensures any UI or code follows the project’s design system and accessibility standards. For example, front-end components should use predefined library components, maintain responsive design (mobile-first, 375px baseline), and meet WCAG accessibility guidelines. Visual consistency (spacing, typography, color usage) is enforced according to `conventions.md`. Micro-interactions and animations should be optimized for performance (using GPU-accelerated CSS, respecting “reduced motion” user settings, etc.). The agent is instructed to generate **“lovable” design** – meaning modern, user-friendly interfaces adhering to best practices (atomic components, type-safe code, error boundaries for new flows, etc.). These

guidelines are baked into the execution phase (especially if generating UI code) and are validated during the Review phase (e.g., running accessibility audits with tools like axe-core).

- **Testing & Quality Assurance:** The agent employs rigorous testing practices across phases. In **Execution**, it follows **Test-Driven Development (TDD)**: writing tests (unit tests for logic, integration tests for end-to-end flows, visual regression tests for UI, performance tests for web vitals, accessibility tests) *before* or alongside implementing functionality ²⁵. The goal is >80% unit test coverage and to catch issues early. Continuous integration checks (linting, type-checking, running all test suites) are part of the process. In **Review & Reflection**, any bugs found result in new tests (regression tests) being added to prevent recurrence ⁴⁴. The instructions also mention using tools (like static analysis, code linters, performance profilers) to ensure code quality and efficiency. All these ensure the final product is robust and production-ready.
- **Multi-Agent Orchestration:** The architecture allows for multiple specialized agents to collaborate. In the improved design, a **“workflow gateway”** decides if a predefined workflow (a sequence of sub-agent calls for common patterns) should be executed or if a single sub-agent should handle the task ⁴⁵ ⁴⁶. The system defines various **sub-agents** (context-manager, researcher, coder, tester, etc.) that the main coordinator can invoke as needed ¹² ¹³. In a complex project, the Execution phase can spin up domain-specific sub-agents – e.g., a Frontend Developer agent to build the UI, a Database agent for data schema, a DevOps agent for deployment – and coordinate them. A Mediator or Supervisor agent (the main orchestrator) keeps track of all tasks and ensures they converge. This multi-agent strategy provides modularity and parallelism: different aspects of the project can be handled by experts in those areas, under the orchestration of the main agent. (For example, the “Hierarchical Orchestrator” approach considered having manager agents for Planning, Execution, Review, each overseeing sub-agents in that domain ⁴⁷ ⁴⁸.) The chosen design, however, leans toward a **Workflow Gateway** approach, which achieved high scores in structure, context management, and modularity ⁴⁹. This means the system routes tasks through a central gateway that either triggers a known workflow or delegates to an appropriate sub-agent, ensuring consistency in how tasks are executed.
- **Context Management (Context Engineering):** Because the agent accumulates a lot of information (conversation history, research data, events), it must manage the context size. The design stresses *context engineering* techniques: **Write, Select, Compress, Isolate** ⁵⁰ ⁵¹. The agent writes important information to external memory (files or graph) instead of keeping everything in the prompt. It selects only relevant pieces of context to load at each step (possibly using the knowledge graph or embeddings to retrieve relevant facts, rather than entire logs) ⁵². It compresses/summarizes long histories when needed (for example, summarizing event-stream logs or older research notes once they grow large). And it isolates the context of sub-agents – each sub-agent might use its own scratchpad and only share necessary outcomes, to avoid “context poisoning” between agents ⁵³. In practical terms, at the start of a new phase or when context length is too high, the agent could create a summary of prior events and use that going forward, thus pruning older, less relevant details. This ensures the agent stays within the model’s token limit and remains focused, addressing the **“context bloat”** issue that could otherwise occur.
- **Documentation and Knowledge Preservation:** Documentation is tightly integrated into the process. Key files maintained include:
 - `planning.md` – the evolving blueprint of the project plan (phases, tasks, design decisions).
 - `todo.md` – a checklist of tasks (with sub-tasks) that get checked off as completed.
 - `event-stream.md` – a chronological log of events for traceability.
 - `doc-ref.md` – an index of other documentation and references (including archived docs).

- Artifacts like `RESEARCH.md`, `TOT.md`, `BRAINSTORM.md`, and final prompts are also maintained in an `artifacts/` folder. After each significant change or completed task, these documents are updated to reflect the new state ³⁰ ⁵⁴. The agent is instructed to **never leave placeholders or sections incomplete** – outputs should be finalized before marking tasks done ⁵⁵. Documentation from previous iterations is archived with timestamps so nothing is lost (e.g., old plans moved to `docs/archive/` with date-stamped filenames) ⁵⁶. This comprehensive documentation serves both as a communication mechanism to the user and as an internal knowledge base for the agent's memory.

All the above features work in concert, making the system **highly structured and thorough**. It covers everything from understanding requirements to coding, testing, reviewing, and delivering – essentially aiming for an autonomous software development team embodied in an agent. Next, we map these elements in a hierarchical “tree-of-thought” format to visualize their relationships and the overall process flow.

Hierarchical “Meta” Tree of the Entire Process (JSON & Markdown)

Below is a deep hierarchical representation of the agent's process and information architecture. It captures phases, modules (sub-agents), cross-cutting concerns, and their relationships in a nested structure. First, we present a JSON schema-style outline, followed by a Markdown tree for readability.

```
{
  "Main_Agent_Process": {
    "phases": [
      {
        "name": "Context Gathering",
        "goal": "Understand user intent; collect and summarize all relevant context 57.",
        "triggers": ["Session start", "New user request", "Outdated context"],
        "steps": [
          "Parse user message for explicit & implicit goals 58.",
          "Identify critical missing details; if something essential is missing, ask the user (otherwise do not interrupt).",
          "Load core context files (`context/` directory: planning.md, todo.md, etc.) and recent artifacts (research, brainstorm notes) as needed 14.",
          "Apply context engineering to include only relevant history (via context_engineering_module).",
          "Query the knowledge graph for any entities related to this request (context7.search_nodes).",
          "Summarize gathered context and goals into artifacts/RESEARCH.md (context summary section), with citations and graph references 59.",
          "Log events: UserMessage (for user query), KnowledgeCapture (for context summary or any new facts).",
        ],
        "outputs": [
```

```

    "Clear restatement of user goals.",
    "Concise context summary (saved in RESEARCH.md).",
    "Updated knowledge graph with any new entities (e.g., new Task or Requirement nodes for this request).",
  ],
  "next_phase": "Analysis (once context is sufficiently understood)",
  "module_used": "context_module (Context-Manager sub-agent) 60 ",
},
{
  "name": "Analysis",
  "goal": "Map out the problem domain and structure through entities and relationships 61 .",
  "triggers": ["After Context is gathered", "When needing to organize knowledge"],
  "steps": [
    "Identify key entities (concepts, requirements, tasks, roles, etc.) and relationships from context and goals 15 .",
    "Create a hierarchical diagram (tree-of-thought or graph) linking these entities and showing dependencies or containment.",
    "Update the knowledge graph: create entity nodes for each key item and relation edges (e.g., Task depends_on Requirement, Feature implements Requirement).",
    "Save the diagram in artifacts/TOT.md (preferably in Mermaid or indented list format) 62 .",
    "Log events: Analysis (for the mapping activity), KnowledgeCapture (for each new relation or entity captured).",
  ],
  "outputs": [
    "A comprehensive entity-relation map (tree-of-thought diagram) in TOT.md.",
    "New knowledge graph entries representing the discovered structure."
  ],
  "next_phase": "Research & Synthesis (once the problem is well-structured)",
  "module_used": "analysis_module (Analysis sub-agent) 63 ",
},
{
  "name": "Research & Synthesis",
  "goal": "Gather evidence and best practices from authoritative external sources 64 .",
  "triggers": ["Open questions or knowledge gaps identified", "Complex domain requiring information"],
  "steps": [
    "Formulate search queries for each key entity or unknown using a research-agent (could utilize web search, documentation lookup, etc.) 16 .",
    "Use safety-compliance checks on results (verify information from multiple reputable sources; avoid unverified snippets) 17 .",
    "Extract relevant facts, guidelines, examples, and solutions. For

```

```

each, record it with a source citation and link to related entities (e.g.,
attach to requirements in the graph).",
    "Summarize findings and insights in artifacts/RESEARCH.md (research
notes section), organized by topic or entity, with citations 17.",
    "Update knowledge graph with new information nodes or attributes
(e.g., an entity 'PerformanceBudget' discovered would be added, or a relation
'Requirement X informed_by Source Y').",
    "Prioritize the info by relevance and authority (could mark some
findings as high priority for design). Possibly filter or weigh the evidence in
the graph 65.",

"Log events: KnowledgeCapture (for each significant fact learned, with
DataSource metadata), PlanUpdate if any new requirement or constraint is
discovered that affects planning."
    ],
    "outputs": [
        "Research report with cited evidence (appended to RESEARCH.md).",
        "Enriched knowledge graph linking sources to design elements.",
        "Any updated constraints or requirements identified (fed into
planning).",
    ],
    "next_phase": "Brainstorm & Evaluation (once enough info is gathered to
generate ideas)",
    "module_used": "research_module (Research sub-agent) 66 "
  },
  {
    "name": "Brainstorm & Evaluation",
    "goal": "Generate diverse solution ideas or prompt components and
evaluate them 18.",
    "triggers": ["After research, when multiple approaches are possible",
"Design phase needs creativity"],
    "steps": [

"Brainstorm all potential modules, features, or strategies relevant to the task
(divergent thinking) 19. E.g., different architectures, libraries, or even
whether to use a multi-agent approach.",
    "Consider enhancements like Graph-of-Thought or hierarchical teams if
relevant (the agent can propose these if they add value) 19.",
    "For each idea, evaluate its relevance, feasibility, impact, and
alignment with requirements. Possibly score each 1-10 and note pros/cons 67.",
    "Document all ideas and their evaluations in artifacts/BRAINSTORM.md,
ranking them by score 67.",
    "Select the top-ranked ideas (or a combination thereof) to carry
forward into planning. Justify why these were chosen (e.g., highest impact and
feasible).",
    "Log events: Analysis (for the evaluation process), PlanUpdate (to
denote that certain approaches will be used in the plan).",
    ],
  },

```

```

    "outputs": [
      "A ranked list of solution approaches with scores and rationale (BRAINSTORM.md).",
      "Selected approach(es) to implement, which will inform the plan."
    ],
    "next_phase": "Planning (once one or more approaches are selected)",
    "module_used": "brainstorm_module (Brainstorming agent) 68 "
  },
  {
    "name": "Planning",
    "goal": "Formulate a concrete execution plan with tasks, timeline, and responsibilities 20 .",
    "triggers": ["After choosing approach", "Scope change requires re-planning"],
    "steps": [
      "Synthesize inputs (goals, context, research insights, chosen design) into an actionable plan 21 .",
      "Break project into phases and tasks. Group tasks under the lifecycle phases (or functional categories) they belong to 3 . Ensure coverage of all requirements and edge cases.",

      "Order the tasks logically, define dependencies (using knowledge graph relations like Task depends_on another Task/Requirement).",
      "Write the plan in @context/planning.md as a numbered list of steps/tasks, with details for each 69 . Also update @context/todo.md as a checklist of tasks (with sub-tasks if needed).",
      "Assign each task to an agent or module capable of executing it. For example, label tasks that are front-end related vs back-end, etc., or note if a particular workflow is applicable.",
      "Review the draft plan with a panel of virtual expert perspectives (simulate feedback from requirements expert, architect, designer, etc.) 22 . For each perspective, consider if the plan meets their criteria (e.g., architect might suggest an additional scalability task).",
      "Incorporate expert feedback: adjust tasks, add missing ones (e.g., add \"Setup CI pipeline\" if the DevOps expert expected it), remove unnecessary ones, refine priorities 22 .",
      "Update planning.md and todo.md with any changes. Persist the plan to memory (e.g., memory.store with key 'plan-[date]'). Possibly take a snapshot of the knowledge graph now that tasks are defined (for traceability).",

      "Log events: PlanUpdate (for each change or when final plan is ready), KnowledgeCapture (to note any best-practice or requirement added during planning).",
    ],
    "outputs": [
      "Finalized project plan in planning.md (phases, tasks with details).",
      "Populated task checklist in todo.md (with each task to be done/checked).",
    ]
  }
]

```

```

    "Knowledge graph updated with Task entities and their relations
(belongs_to Phase, uses Module, depends_on Requirements, etc.).",
    "Ready to proceed with Execution phase tasks."
  ],
  "next_phase": "Execution (once the plan is agreed and tasks listed)",
  "module_used": "planning_module (Planning-Task agent) 70 "
},
{
  "name": "Execution",
  "goal": "Implement the tasks to build the solution, maintaining quality
and context fidelity 71 .",
  "triggers": ["Approved plan with pending tasks", "Resume after a pause
with tasks remaining"],
  "steps": [

    "For each task in @context/todo.md (starting from the highest priority or first
in sequence):",
      "- Determine if the task matches a known workflow (check workflows/
WORKFLOWS.md). **If yes**, invoke that workflow (which may call multiple sub-
agents in sequence) 45 . **If no**, use the invocation-chain-generator to suggest
the optimal sequence of sub-agent calls to accomplish the task 45 72 .",

    "Invoke the appropriate sub-agent or sequence for the task (e.g., call
execution-agent for coding tasks, which itself might call domain-specific
agents). Ensure each sub-action is logged.",
      "Within each task execution: follow best practices per cross-cutting
modules:",
        "- **Design**: Adhere to design system and conventions (use shadcn/UI
components, 8pt grid, responsive design, etc.).",
        "- **Coding standards**: Follow conventions.md for naming and style;
keep functions ≤50 lines, ensure TypeScript strict mode if applicable.",
        "- **Security**: Validate inputs, handle errors, avoid secrets in
code, etc., as per security_module.",
        "- **Testing**: Write tests before or alongside code for TDD. After
coding, run all relevant tests (unit tests, integration, end-to-end, performance
budgets, accessibility audits) and ensure they pass 25 .",

        "- **Documentation**: Update code comments, architecture docs, or user docs if
appropriate for the task.",
          "Capture the result of each tool action: if editing code, check the
diff; if running tests, gather results; if deploying, note the outcome. Log an
Observation event with key details.",
          "If a test fails or an issue is detected, log an Error event and
attempt a fix immediately (go back to code step, then re-run tests). Only
proceed when the task's acceptance criteria are satisfied (tests pass, quality
checks OK).",
          "Mark the task as completed in todo.md (check it off) 73 . Log a
PlanUpdate event to reflect progress.",

```

"Synchronize context: update planning.md if any task result changes the plan (e.g., a simpler solution eliminated the need for a later task). Add any new knowledge to the graph (e.g., component X implemented, link it to Module entity). Optionally, snapshot the event-stream or graph state.",

"Move to the next task and repeat the execution loop. If new tasks are discovered during execution (for example, fixing a bug reveals another necessary fix), add them to todo.md and planning (PlanUpdate) and possibly adjust phase progression (this could temporarily pause Execution to handle a mini-Planning for the new task).",

"Log events continuously: Action (tool usage), Observation (outcomes), PlanUpdate (task done or new task added), KnowledgeCapture (any new insights).",

"Loop Condition: Continue until ****all tasks in todo.md are completed**** for the current project ³³. If a contradiction or missing piece of info is found that blocks execution, pause and return to Research or Planning phase as needed ²⁷."

```
],
  "outputs": [
    "All planned features/requirements implemented in code or configuration.",
    "Test results indicating the solution meets quality benchmarks (all tests passing, performance within limits, etc.).",
    "Updated codebase, database (if applicable), and other artifacts reflecting the changes.",
    "Documentation (code comments, README, user guide if needed) updated to reflect the new system."
  ],
  "next_phase": "Review & Reflection (once no tasks remain unchecked and solution is built)",
  "module_used": "execution_module (Execution agent, coordinating dev sub-agents) 24 "
},
{
  "name": "Review & Reflection",
  "goal": "Assess and refine the solution using expert feedback and self-critique 28.",
  "triggers": ["All tasks done (or periodically during execution for interim reviews)", "After major milestones in execution"],
  "steps": [
    "Gather a panel of simulated expert reviewers (requirements analyst, software architect, QA tester, UX designer, product manager, domain expert, etc.) via the review-reflection-agent 28.",
    "For each perspective, systematically evaluate the output:",
    "- **Requirements**: Does the solution meet all user requirements and acceptance criteria? Any missing use-cases or non-functional requirements (performance, security)?",
```

```

"- **Architecture**: Is the code/design clean, scalable, maintainable? Are there
any architectural anti-patterns or technical debt introduced?",
    "- **Performance**: Do performance tests show it meets targets (e.g.,
web vitals)? Any slow parts or memory issues?",
    "- **Tooling & DevOps**: Are CI/CD pipelines in place and passing? Is
the dependency management up to date? Any security alerts from scanners?",
    "- **Design/UX**: Is the UI/UX aligned with best practices and the
design system? Any accessibility issues or poor user flows?",
    "- **Product fit**: Does the output solve the user's core problem
effectively? Is anything over-engineered or under-scoped?",
    "- **Domain-specific**: (If applicable) Does the solution respect
domain constraints (e.g., healthcare data compliance, finance regulations)?",
    "Aggregate the feedback from all these 'experts'. Identify any
**critical issues** that must be fixed vs. minor suggestions that could be
deferred 74 .",
    "For each critical issue found, create a plan to fix it: possibly
spawn new tasks (e.g., \"Refactor module X for clarity\", \"Improve error
handling for Y\") and go back to Execution for those tasks. Mark these in
todo.md (and link to bug tracking if used).",
    "Implement fixes for critical issues immediately (return to Execution
loop for each fix). After fixes, re-run tests/reviews for validation. Minor
issues can be recorded in documentation or backlog for future improvement.",
    "Reflect on the development process itself: the agent considers what
could be improved in its own approach (metacognition). It logs lessons learned
or patterns observed as KnowledgeCapture events (e.g., \"Next time, allocate
more time to performance testing\" or update conventions/best practices).",
    "Update all documents: planning.md may be updated to mark phases
complete; a post-mortem or summary of improvements might be added to docs; the
knowledge graph may log observations of final metrics or known bugs.",
    "Loop Condition: Repeat the review → fix → review cycle until **all
critical issues are resolved** and the output meets quality standards and user
expectations 75 . Ensure no high-severity bugs remain.",
    "Log events: Analysis (for review findings), PlanUpdate (for any new
tasks/bugs added or resolved), KnowledgeCapture (for lessons learned or best
practices captured), Error (if any new errors encountered during review
processes).",
  ],
  "outputs": [
    "Refined solution with enhanced quality (post-review).",
    "List of any remaining known issues (all minor or acceptable).",
    "Lessons learned documented (for continuous improvement).",
    "Confidence that the solution is production-ready and meets all
requirements."
  ],
  "next_phase": "Delivery (once the team is satisfied with the solution)",
  "module_used": "review_module (Review-Reflection agent) 76 "
},

```

```

{
  "name": "Delivery",
  "goal": "Present final outputs to the user and ensure closure of the
project 32.",
  "triggers": ["All phases complete and verified", "User asks for
results"],
  "steps": [
    "Double-check that all tasks in todo.md are completed (no checkboxes
left) and planning.md is up-to-date with final status 33.",
    "Ensure no critical issues remain unaddressed from Review; if any are
found at the last minute, handle them (possibly loop back to Execution/Review
briefly).",
    "Gather all deliverables to provide to the user. This typically
includes:",
    "- The improved agent prompt (CLAUDE.md) and process
(CLAUDE_PROCESS.md) in full 77.",
    "- Key artifacts: RESEARCH.md (context and research summary), TOT.md
(tree-of-thought diagram), BRAINSTORM.md (ideas and scores), planning.md (final
plan), todo.md (with all items checked, possibly providing a history of what was
done).",
    "- Any code or configuration files that were created (if the user
needs the actual product deliverables). Possibly provide a link to a repository
or attachments of code files.",
    "- Simulation results or additional analysis if requested (e.g.,
performance reports, test logs).",
    "Attach these files using the `computer.sync_file` tool 34, ensuring
the user can access each relevant artifact.",
    "Compose a final summary message to the user: explain what was
accomplished, highlight key choices or results, and ensure the user knows the
objectives were met. Keep it professional and clear, without exposing internal
chain-of-thought or any sensitive info.",
    "Log a Delivery event (with timestamp and summary of delivered
items).",

    "Transition to a **standby state**, meaning the agent will stop active
operations and wait for any new user input. No further actions are taken until a
new instruction arrives 36."
  ],
  "outputs": [
    "All final deliverable files provided to the user (attachments or
links).",
    "A closing report/message summarizing the project outcome.",
    "The agent's internal state saved (memory and knowledge graph
snapshots, etc.) in case of future re-engagement.",
    "Agent enters idle mode, ready for further requests."
  ],
  "next_phase": "N/A (this is the last phase, then the process ends or
awaits new user input)",

```

```

    "module_used": "delivery_module (Delivery agent) 78 "
  }
],
"crosscutting_components": {
  "Memory & Knowledge Graph": {
    "role":
"Persist important info, link entities and tasks, enable context recall 38 .",
    "mechanisms": [
      "Stores plan, research, etc. via memory.store with keys.",
      "Graph with nodes: Task, Phase, Module, File, Requirement,
Observation, etc. 3 .",
      "Graph with relations: depends_on, belongs_to, implements, uses,
assigned_to, logs (for observations) 37 .",
      "Used for context retrieval to avoid forgetting or hallucinating."
    ]
  },
  "Logging": {
    "role": "Record every step and decision in event-stream.md 56 .",
    "events": ["UserMessage", "PhaseChange", "ModuleSelection", "Action",
"Observation", "PlanUpdate", "KnowledgeCapture", "Error", "Delivery 40 6 "],
    "purpose":
"Traceability, debugging, learning (to reflect on process and improve).",
  },
  "Guardrails (Safety)": {
    "role": "Ensure safe, ethical, and valid operations across all phases
43 .",
    "measures": [
      "Verify external info (don't trust unverified content) 42 .",
      "Sanitize commands/outputs to prevent injections.",
      "No side-effect actions (like purchases or deployments) without user
confirmation 6 .",
      "Privacy: no leaking secrets or sensitive data.",
      "Regulatory compliance (GDPR, HIPAA, etc.) checks."
    ]
  },
  "Design & UX": {
    "role": "Enforce design system and good UX in all user-facing outputs.",
    "guidelines": [
      "Use approved UI components (shadcn/ui) or standard patterns.",
      "Responsive design from mobile upward.",
      "Accessibility (WCAG AA) and intuitive UX flows.",
      "Atomic components, TypeScript strict, error boundaries, etc."
    ]
  },
  "Testing & QA": {
    "role": "Maintain high quality through testing at all levels.",
    "practices": [
      "TDD: write tests before code 25 .",

```

```

        "Maintain >80% unit test coverage.",
        "Integration tests for key flows; visual regression tests for UI;
performance tests for speed/memory.",
        "Automate tests in CI pipeline; do not proceed if tests fail."
    ]
},
"Security": {
    "role": "Protect security and data integrity throughout.",
    "checks": [
        "Validate all inputs (use schemas like Zod).",
        "Use parameterized queries, escape outputs to prevent XSS/SQLi.",
        "Store secrets in env variables, never commit them.",
        "Implement authZ/authN where needed (e.g., route guards).",
        "Regular dependency audits (npm audit, etc.) and fix vulnerabilities."
    ]
},
"Collaboration (Multi-agent)": {
    "role": "Coordinate multiple agents when needed for efficiency or
specialization.",
    "setup": [
        "Workflow gateway decides if a predefined multi-step workflow should
be run 79 .",

        "Coordinator can delegate to sub-agents (e.g., a specialized writer or coder)
and wait for their result.",
        "Supports parallel tasks via separate agents (with a mediator to
sync).",
    ]
}
}
}
}
}

```

Markdown Tree Representation: Below is the same structure in a Markdown nested list form, which may be easier to read. It outlines the **process hierarchy**, including phases, their triggers/loops, steps, outputs, and relationships to modules and crosscutting components:

- **Main Agent Process**

- **Phase: Context Gathering** 57

- *Goal:* Capture user intent and all relevant context (files, history, knowledge) at session start.
- *Triggers:* Session start or reset; user provides new instructions that require fresh context.
- *Key Steps:*
 - Parse the user's message for goals and requirements 58 .
 - Identify any missing info; ask clarifying questions if absolutely necessary.
 - Load core context files (`planning.md` , `todo.md` , `event-stream.md` , etc.) and recent artifacts (research notes, etc.) 14 .
 - Use context engineering to trim irrelevant history and keep context window manageable 14 .

- Query knowledge graph for related items (e.g., existing tasks or domain info).
- Summarize the situation and user goal in a **Context Summary** (saved to RESEARCH.md) with citations ⁵⁹.
- Log events: **UserMessage** (user's query), **KnowledgeCapture** (context captured).
- *Loop/Conditions*: Repeat if goals are still unclear or new documents are provided ⁸⁰. Otherwise, proceed to Analysis.
- *Outputs*: Clear statement of user objectives; updated context in memory and RESEARCH.md; knowledge graph seeded with new entities.
- *Module/Sub-agent*: `context_module` (Context Manager agent) handles this phase ⁶⁰.
- **Phase: Analysis** ⁶¹
 - *Goal*: Develop a structured understanding by mapping out entities and their relationships.
 - *Triggers*: Completion of context gathering; entering a complex problem space that needs organization.
 - *Key Steps*:
 - Identify key **entities** (requirements, tasks, users, components, etc.) and how they relate ¹⁵.
 - Construct a **tree-of-thought** or **graph-of-thought** diagram visualizing these relationships (hierarchies, dependencies, flows).
 - Update the **knowledge graph** with these entities and relations (e.g., Task nodes linked to Requirement nodes, etc.) ³.
 - Save the diagram in `TOT.md` (using Mermaid or nested lists for clarity) ⁶².
 - Log events: **Analysis** (for conceptual mapping), **KnowledgeCapture** (each new entity or relation captured).
 - *Loop/Conditions*: This mapping can be updated later if new entities surface (e.g., in Research or Planning) ⁸¹. Otherwise, move to Research.
 - *Outputs*: An **entity-relation map** of the problem (TOT.md) and a populated knowledge graph schema for the project.
 - *Module/Sub-agent*: `analysis_module` (Analysis agent) for mapping and graph updates ⁶³.
- **Phase: Research & Synthesis** ⁶⁴
 - *Goal*: Collect evidence, examples, and best practices from external sources to inform solution design.
 - *Triggers*: Open questions or uncertainties from analysis; need for domain knowledge or technical references.
 - *Key Steps*:
 - Use a `research_agent` to search docs, APIs, or web for information on each key entity or requirement ¹⁶.
 - Utilize `safety_compliance_agent` to validate and cross-check external info to avoid misinformation ¹⁷.
 - Extract relevant insights (e.g., "Use OAuth2 for auth", "Recommended architecture for real-time apps") with citations.
 - Write a **Research Summary** in `RESEARCH.md` (continuing from context summary) with organized notes and references ¹⁷.
 - Link insights to corresponding entities in the knowledge graph (for example, tag a Requirement node with an Observation about a regulation found).
 - Prioritize info by relevance: mark which findings are critical vs optional, possibly weighting edges in the graph ⁶⁵.
 - Log events: **KnowledgeCapture** (for each key fact, including its source), maybe **PlanUpdate** if new requirements or constraints are discovered.

- *Loop/Conditions:* If conflicting info or gaps remain, continue searching until resolved ⁸² . Otherwise, proceed to Brainstorm.
- *Outputs:* An **evidence-based knowledge base** (RESEARCH.md with citations) and updated context for ideation; graph enriched with factual nodes/edges.
- *Module/Sub-agent:* `research_module` (Research agent) for external info gathering ⁶⁶ .
- **Phase: Brainstorm & Evaluation** ⁸³
 - *Goal:* Imagine various solution approaches or prompt improvements and evaluate their merit.
 - *Triggers:* Sufficient information available (post-research) to consider multiple ways forward; complex design space that benefits from creative thinking.
 - *Key Steps:*
 - Brainstorm a wide range of ideas: different architectures, algorithms, UX designs, agent collaboration strategies, etc. ¹⁹ .
 - Optionally include innovative strategies (e.g., “use a hierarchical agent team” or “apply Graph-of-Thought technique”) if relevant ¹⁹ .
 - For each idea, **evaluate** it: relevance to the problem, expected effectiveness, difficulty, risks. Score each (e.g., 1–10 or low/med/high) ⁶⁷ .
 - Record all ideas and their evaluations in `BRAINSTORM.md` , with a ranking of top candidates ⁸⁴ .
 - Choose the best idea(s) or combine several into a proposed approach. Justify the selection (e.g., highest impact with acceptable effort).
 - Log events: **Analysis** (for the evaluation process itself), **PlanUpdate** if the choice impacts the plan structure.
 - *Loop/Conditions:* If a new idea emerges later (during planning/execution), revisit brainstorming to evaluate it ⁸⁵ . Otherwise, proceed with chosen approach to Planning.
 - *Outputs:* **Ranked list of ideas** with pros/cons (BRAINSTORM.md) and a clear decision on which approach to implement.
 - *Module/Sub-agent:* `brainstorm_module` (Idea generation agent) for creative ideation ⁶⁸ .
- **Phase: Planning** ⁸⁶
 - *Goal:* Translate the chosen solution approach into a concrete project plan with tasks, phases, and deliverables.
 - *Triggers:* An approach/solution is decided; or scope changes require re-planning mid-project.
 - *Key Steps:*
 - Consolidate knowledge (goals, context, research insights, and chosen idea) into a structured **project plan** ²¹ .
 - Break down the solution into **tasks**. Organize tasks by phases (e.g., Research tasks, Setup tasks, Frontend tasks, Testing tasks, etc.) ³ .
 - Define task details: what needs to be done, success criteria, and which module/agent is responsible.
 - Note dependencies: Task A depends_on Task B, etc., and record these in the knowledge graph and in task descriptions.
 - Populate `planning.md` with an ordered list of all tasks/steps required (essentially a roadmap from start to finish) ⁶⁹ . Include any design diagrams or architecture decisions if needed.
 - Populate `todo.md` with a checklist of these tasks (possibly nested sub-tasks), all initially unchecked ⁶⁹ .

- Engage **expert review** of the plan: simulate feedback from various expert viewpoints (requirements check, architecture soundness, etc.) ²² . For example, a security expert might suggest adding a penetration test task.
- Refine the plan based on feedback: add/remove tasks, adjust priorities, clarify acceptance criteria ⁸⁷ .
- Ensure the plan is **realistic and complete** (covers all requirements and quality checks).
- Log events: **PlanUpdate** (every time tasks are added, removed, or reordered), culminating in a PlanUpdate for final plan approval.
- *Loop/Conditions*: If new requirements come in or experts reveal gaps, update the plan (which may loop back to adding tasks or even doing more research) ⁴ . Otherwise, proceed to Execution.
- *Outputs*: A **detailed project plan** (planning.md) and a corresponding **task board** (todo.md); both synced with each other and stored in memory. Knowledge graph now contains Task entities linked to Phases and Modules ³ .
- *Module/Sub-agent*: `planning_module` (or planning-task-agent) for plan creation ⁷⁰ .
- **Phase: Execution** ⁷¹
 - *Goal*: Execute the plan's tasks to build the actual solution, while maintaining adherence to quality and context.
 - *Triggers*: A set of pending tasks is ready (plan approved); or resuming after a break with tasks remaining.
 - *Key Steps*:
 - **Task Loop**: For each task in `todo.md` (in order, or by priority):
 - Check if there's a predefined **workflow** for this task type (see `WORKFLOWS.md`) ⁴⁵ . If yes, execute that workflow (which might sequentially invoke multiple sub-agents).
 - If no workflow matches, use the **invocation-chain-generator** to decide the best sequence of sub-agent calls to accomplish the task ⁴⁵ .
 - Invoke the needed sub-agent(s) to perform the task. For example, a "Write API endpoint" task might involve: call `execution_agent` which uses the `developer-backend` agent to write code, then `testing-QA` agent to run tests. Each invocation is a tool action logged as an **Action** event.
 - Monitor the outcome of each action (**Observation** events). If the sub-action fails (e.g., tests fail), handle it: maybe fix code and retry (log **Error** events on failures) ²⁷ .
 - Apply cross-cutting standards on the fly: ensure code follows conventions, run linters, ensure design consistency, etc., as part of doing the task.
 - Once the task's acceptance criteria are met (e.g., feature works and tests pass), mark it **completed** in `todo.md` (check it off) ⁷³ .
 - Update relevant docs: maybe update planning.md to note completion or any change (like "Module X implemented"). Also update the knowledge graph (e.g., mark task node as done, add relation that a certain code file was produced).
 - Log a **PlanUpdate** event for task completion, and capture any key knowledge (like "We learned library Y is needed, added to dependencies") as KnowledgeCapture.
 - Continue to the next task. If executing a task reveals a new necessary task (scope creep or bugfix), add it to the plan (PlanUpdate) and possibly pause to reprioritize.
 - Maintain context: after each task or sub-task, sync the context files (so other agents or later steps see up-to-date info) ²⁶ .
 - *Loop/Conditions*: Keep iterating through tasks until **todo.md is empty (all tasks done)** ³³ . If a blocking issue arises (e.g., missing info or a design flaw), the agent may pause Execution to

go back to Research or Planning to address it ²⁷ . If errors occur, attempt to fix up to one time; if still failing, consider escalating (maybe ask user or mark as bug) ⁸⁸ .

- *Outputs:* A working **implementation of the project** – code written, features built, tests passing. All artifacts (code, configs, etc.) are produced as per the plan. Documentation and knowledge graph are updated to reflect the final build.
- *Module/Sub-agent:* `execution_module` (Execution coordinator agent invoking coding/testing sub-agents) ²⁴ .

• Phase: Review & Reflection ²⁸

- *Goal:* Validate the solution against all requirements and quality standards; reflect on improvements.
- *Triggers:* All planned tasks are done (or at a major checkpoint); preparing for final delivery but first doing a thorough check.
- *Key Steps:*
 - Simulate a **multi-expert review**: for each expert lens (requirements, architecture, performance, security, UX, etc.), review the product. This can involve running additional tools: security scanners, performance benchmarks, UX heuristic evaluations, etc. ²⁹ .
 - Compile a list of findings: e.g., any **bugs** found, performance bottlenecks, UI quirks, unmet requirements. Categorize them by severity.
 - Address **critical issues** immediately: create new tasks for fixes and go back to Execution for each fix. For example, if accessibility tests reveal missing alt text, create a task “Add alt text to images” and fix it. Only critical must-fix issues are tackled now; minor issues can be noted for later.
 - As fixes are implemented, re-run relevant tests or reviews to ensure the issues are resolved.
 - Conduct a **post-mortem reflection**: the agent examines its own process, noting what worked well and what could be improved next time (e.g., “spent too much time in research phase, could streamline that”). Document these as “lessons learned.”
 - Log events: **Analysis** (for the review analysis), **PlanUpdate** (if new tasks/bugs are added or when resolved), **KnowledgeCapture** (for lessons learned or important observations like final performance metrics), **Error** (if any new error encountered during review).
 - *Loop/Conditions:* Repeat review and fix cycles until **all critical issues are resolved** and the solution meets acceptance criteria ³¹ . Minor issues or nice-to-have enhancements might remain but are acceptable for delivery (could be logged for future). If any change re-opens a previous phase necessity (e.g., a new requirement emerges), the agent might loop back to Planning (scope change). Otherwise, proceed to Delivery.
 - *Outputs:* A **polished final product**, with any major defects fixed. A set of **lessons learned or documentation updates** about the process. Possibly an updated plan indicating any changes made during review (and that all items are complete).
- *Module/Sub-agent:* `review_module` (Review-Reflection agent and possibly domain-specific reviewers) ⁷⁶ .

• Phase: Delivery ³²

- *Goal:* Hand over the finished output to the user and conclude the project.
- *Triggers:* Solution passes review (or user asks for whatever is ready); end of development cycle.
- *Key Steps:*

- Verify everything is truly done: check `todo.md` for any open tasks; confirm all phases completed, all requirements met ³³. If something is unfinished, go back to the relevant phase to complete it before proceeding.
 - Gather all **deliverables**: compiled code or application, documentation files (`RESEARCH.md`, `TOT.md`, `BRAINSTORM.md`, etc.), updated prompts (the agent's own definitions), logs if requested, etc.
 - Use `computer.sync_file` or similar to attach these files for the user ³⁴. Ensure no file is missed (the instructions explicitly list `RESEARCH.md`, `TOT.md`, `BRAINSTORM.md`, `PROCESS_MAPS.md`, `planning.md`, `todo.md`, and others to attach) ³⁴.
 - Present the final **agent prompt and process description** clearly (the improved `CLAUDE.md` and `CLAUDE_PROCESS.md`) as part of the answer, with all sections filled – no placeholders ³⁵.
 - Provide a summary in the response: recap what was accomplished, any notable design choices, and possibly next steps or suggestions for the user. Keep tone professional and helpful.
 - Log a **Delivery** event and then transition to **standby** mode ³⁶. Standby means the agent stops active processing and simply waits for any new user input, ensuring no unintended actions occur while idle.
 - *Loop/Conditions*: If while preparing deliverables an inconsistency is found (e.g., a file out of sync), fix it (even if it means briefly re-opening Execution or Review) before finalizing ⁸⁹. Otherwise, once delivered, the project loop ends.
 - *Outputs*: All final output files delivered to the user; a closing message. The agent's state is saved for potential future use, and it becomes idle.
- **Cross-Cutting Modules & Systems**: *(These operate across multiple phases rather than being a single phase.)*
 - **Memory & Knowledge Graph** – Persistent context store and relational graph that **records tasks, entities, and their links** ³. Every phase updates this: e.g., new Task nodes in Planning, Observation nodes in Review. It enables context recall (via queries) and prevents forgetting key info. Memory snapshots allow the agent to “remember” past sessions.
 - **Event Logging** – A consistent logging mechanism captures events in `event-stream.md` with timestamps ⁵⁶. This provides **traceability** for the entire process, useful for debugging or auditing the agent's decisions. Logged events include user messages, tool actions, observations, errors, etc., but *exclude* the agent's hidden chain-of-thought to maintain privacy ⁶.
 - **Guardrails (Safety & Ethics)** – A set of rules always in effect to **prevent harmful actions or hallucinations**. For example, the agent must verify external data from multiple sources before trusting it ¹⁰, ignore any malicious instructions from web content, and ask user permission before any side-effect (like sending emails or making purchases) ⁶. The guardrails also enforce not exposing private thoughts or confidential info in outputs, and sanitizing any code or command execution.
 - **Design & UX Guidelines** – Standards that ensure any created UI or content is **high-quality and accessible**. The agent follows established conventions (in `conventions.md` or design system docs): e.g., using the correct color palette, fonts, spacing; ensuring responsive design; adding alt text for images; using ARIA labels for accessibility, etc. These are checked during Execution and verified in Review.
 - **Testing & QA** – A quality-first approach embedded throughout. Tests are written and run as part of Execution for each feature, and additional QA (like manual or exploratory testing via simulated agents or tools) is done in Review. No code is considered “done” until it's tested. Performance

budgets (max allowed load time, etc.) and security scans are also part of QA, potentially generating tasks if they fail.

- **Security & Compliance** – Continuous enforcement of security best practices: input validation, output encoding, secure secret management, permission checks, dependency vulnerability audits, and compliance with any domain-specific regulations. This often overlaps with Testing (e.g., running security test tools) and Guardrails (e.g., not outputting secrets).
- **Workflow & Multi-Agent Orchestration** – The **workflow gateway** sits at the decision point of each action, allowing either a direct sub-agent call or a more complex workflow of many steps ⁷⁹. This mechanism improves flexibility: known sequences (like “set up a new project repo” or “troubleshoot a failing test”) can be pre-defined so the agent doesn’t reinvent the wheel. Multi-agent coordination patterns (like having parallel agents under a supervisor) are possible; the chosen architecture uses a central orchestrator with sub-agents, as it scored well in modularity and context management ⁴⁹.

This hierarchical mapping shows how the **phases** and **modules** interconnect and how cross-cutting systems support them. It illustrates the full workflow from start to finish, including loops and conditional branches (e.g., handling errors or new information). Next, we will walk through the entire process flow step-by-step, mapping each phase to atomic actions, loop conditions, and triggers in a narrative form for clarity.

Full Process Flow Mapped to Atomic Actions and Conditions

Now we trace the **entire agent process** as it would execute, breaking it down into atomic actions and noting how control flows between phases, including any loops, decisions, and triggers:

1. **Start / Initialization:** The agent begins a session (or receives a new user request in an existing session). It **loads its core context** (identity and process definitions like `CLAUDE.md` and `CLAUDE_PROCESS.md`) to ensure it has its instructions and memory available. It then immediately enters the **Context Gathering phase** to handle the user’s request.
2. **Context Gathering Phase:**
3. *Analyze User Input:* The agent reads the user’s message. For example, suppose the user asked: “Design and build a web app that tracks fitness goals.” The agent parses this for explicit goals (“web app for fitness tracking”) and implicit requirements (maybe “user authentication, data visualization, mobile-friendly”).
4. *Check Preconditions:* It considers what information it needs. Are requirements clear? If a critical detail is missing (say the user didn’t specify a platform), the agent would at this point formulate a **clarifying question** to ask the user before proceeding. (If so, it would pause the process waiting for the user’s answer, then resume with updated context.)
5. *Load Existing Context:* The agent opens files in `context/`: for example, it reads `planning.md` and `todo.md` from a previous session, or finds relevant info in `doc-ref.md` (perhaps there’s a similar past project documented it can reference). It uses a **context_engineering** routine to include only the most pertinent parts of these files in the working context (since it might not fit the model’s window if everything is loaded) ¹⁴. For instance, it might include an outline of an earlier similar project from `doc-ref.md` but exclude unrelated sections.
6. *Knowledge Graph Query:* The agent queries its knowledge graph for any nodes related to “fitness app” or previously stored requirements that might match. Let’s say it finds none (first time this is done), so

no existing nodes yet, but it might create a new high-level Task node like “Build fitness tracker app” and a Requirement node for “track fitness goals” to start the graph.

7. *Summarize Context*: It compiles all the above into a concise summary: e.g., “The user wants a fitness goals tracking web application. Context: no prior data exists for this specific domain; relevant design conventions and past patterns will be applied. Likely requirements: user accounts, goal input, progress charts...” – and writes this to the **Research document** as the initial context summary. It cites any sources (if, say, it pulled a standard definition of fitness tracking from somewhere, it would cite that).
8. *Log Events*: It logs a **UserMessage** event (with the content of the user’s request), and a **KnowledgeCapture** event noting the initial understanding of the task (with no external source, or possibly referencing internal files).
9. *Transition*: With goals understood and context set, the agent proceeds to **Analysis**. (If anything was unclear or new context arrived mid-way, it would loop in Context phase, but in this example we assume it’s clear enough to continue.)
10. **Phase Change -> Analysis Phase**: The agent logs a **PhaseChange** event indicating it’s now in Analysis. In this phase, it performs the following atomic actions:
 11. *Entity Extraction*: The agent identifies key concepts from the user’s request and context. Continuing the example, entities might include: **User, Fitness Goal, Progress Tracker, Web App, Database, Frontend UI, Goal Metrics** (and so on). It also notes relationships: e.g., *User* can have many *Fitness Goals*, a *Goal* has *Metrics* (like daily progress), etc. It might also infer needed components: a login system, a progress visualization, etc.
 12. *Update Knowledge Graph*: For each identified entity, it creates a node in the knowledge graph (type could be “Concept” or “Component”). It then creates edges for relationships: e.g., *User -tracks-> Goal*, *Goal -displayed_on-> Dashboard*, etc. If some entities are actually tasks, it might hold off creating Task nodes until Planning – at this stage it’s more conceptual. But it might create a **Requirement** node for each inferred requirement (like “Allow user to log goals” as a requirement).
 13. *Construct ToT Diagram*: The agent translates these entities and relations into a structured diagram. For instance, it might outline a tree like:
 - Fitness Tracker App
 - Users
 - Goals
 - Metrics/Progress
 - Components
 - Frontend (React)
 - Backend (API, Database)
 - Non-functional Reqs
 - Security (authentication)
 - Performance (quick load, handle X users)

This diagram is saved to `TOT.md`. It could use a Mermaid mindmap or a bullet list structure to show how concepts connect.
 14. *Log Events*: It logs an **Analysis** event describing that an entity map was created, and **KnowledgeCapture** for each notable entity/relation (possibly with references if any came from known templates or standards).

15. *Transition*: The agent has now externalized the problem structure. It moves to **Research & Synthesis** to fill in knowledge gaps for the items identified.

16. **Phase Change -> Research & Synthesis**: Logs **PhaseChange**. Atomic actions here include:

17. *Identify Research Questions*: Based on the analysis, the agent decides what it needs more information on. E.g., "What are best practices for tracking fitness goals?", "Are there existing libraries for charting progress?", "Security considerations for health data?", etc.

18. *Execute Searches*: For each question, it uses the `brave_web_search` or other search tools. For instance, it searches for "fitness tracking app best practices" or "open source fitness tracker examples." Each search is a tool **Action**.

19. *Fetch and Read Sources*: The agent clicks on relevant results (e.g., an article on fitness app design, a StackOverflow question about chart libraries, official docs for a chart library). It reads these pages (via tools like `browser.open`) and extracts useful points. For example, it finds: "Use OAuth2 for user login for security", or "Users should be able to set and update goals, and see weekly progress charts", or "Library X can plot interactive graphs on React." Each piece of info is noted.

20. *Verify Information*: The agent cross-checks important facts, especially if they are critical (like security advice) – possibly using the `safety_compliance_agent`. If one source says one thing and another contradicts, the agent might do an additional targeted search or decide which is more credible (preferring official documentation or widely cited sources).

21. *Record Findings*: The agent writes a **Research** section in `RESEARCH.md`. For instance:

- **User Authentication**: Recommend using OAuth2/OIDC for secure login **[source]** . Alternatively, social logins could be considered.
- **Progress Visualization**: Library X (MIT license) provides interactive charts **[source]** .
- **Data Model**: Each user goal entry should have fields A, B, C (citing some design reference).
- **Performance**: Use lazy loading for charts to improve initial load time **[source]** . Each bullet is accompanied by a citation (the agent uses the provided format with cursor references in practice).

22. *Update Knowledge Base*: The agent adds relevant details to the knowledge graph. For example, link the *User* entity to an *Authentication* technique observation, or attach a note to the *Frontend* component about using Library X.

23. *Log Events*: Each distinct nugget is a **KnowledgeCapture** (with *DataSource* pointing to the URL or doc). If any new constraint or requirement emerges (say the research highlights a need for GDPR compliance if storing personal data), the agent logs a **PlanUpdate** (this foreshadows an extra requirement for Planning).

24. *Synthesize & Prioritize*: Finally, the agent distills which findings are most crucial for the approach. For instance, it will mark security and user experience findings as must-haves, while maybe noting optional nice-to-haves separately.

25. *Transition*: With a solid base of facts and ideas, the agent goes to **Brainstorm** to leverage this info creatively.

26. **Phase Change -> Brainstorm & Evaluation**: Logs **PhaseChange**. Atomic actions:

27. *Idea Generation*: The agent, armed with context and research, starts generating ideas. For our example, it might brainstorm things like:
- Tech stack choices: “Use Next.js with Supabase for quick dev” vs “Use a custom Node backend with React frontend”.
 - Feature set: “Include social sharing of goals” or “Add gamification (badges)”.
 - Approach to agent architecture: maybe it even considers “Should I spin up a separate agent to handle UI design while another handles backend?” (i.e., multi-agent parallelism).
 - Prompt-engineering ideas: since this agent’s output is also the prompt, it could consider if the prompt structure needs changes (but likely it’s focusing on the product).
28. *List and Expand Ideas*: For each idea, the agent writes down a brief description. It doesn’t censor at this stage – anything potentially useful is noted. Let’s say it listed 5–6 ideas.
29. *Evaluate Ideas*: It then goes through each idea and evaluates:
- Feasibility (given time/tools),
 - Impact (does it significantly improve the outcome?),
 - Alignment with requirements (does user specifically need/want it?),
 - Elegance or best-practice compliance (is it a clean solution or a hack?).
- It might score each on these criteria or an overall 1–10. For example:
- Idea: Use Next.js – Score 8 (high developer efficiency, good SEO out-of-box).
 - Idea: Gamification – Score 5 (could be nice but not requested, might distract core functionality).
30. *Rank and Select*: It orders the ideas by score and decides which to implement. Likely it picks a **tech stack** idea (Next.js + Supabase) and perhaps a moderate feature like social sharing if it seems feasible, ignoring lower-value ones like gamification for now.
31. *Document Brainstorm*: It writes all this to `BRAINSTORM.md`, with headings for each idea, their score, and rationale. The top picks are clearly marked.
32. *Log Event*: Logs **Analysis** for conducting the brainstorm evaluation, and possibly a **PlanUpdate** to note the chosen approach (like “Decided on Next.js + Supabase stack”).
33. *Transition*: Moves on to **Planning** armed with the chosen approach and all previous context.
34. **Phase Change -> Planning**: Logs **PhaseChange**. Atomic actions in Planning:
35. *Draft Plan Outline*: The agent starts outlining the steps to realize the project. It might use the phase structure as a guide:
1. **Setup Project**: initialize Next.js app, set up Supabase project, etc.
 2. **Design Schema**: define database tables for goals, users.
 3. **Implement Auth**: integrate OIDC for user login.
 4. **Implement Frontend Pages**: home page, dashboard for goals.
 5. **Implement Goal Tracking API**: endpoints to create/update goals.
 6. **Implement Progress Visualization**: use chart library to show progress.
 7. **Testing**: write and run tests (unit, integration).
 8. **Performance & Security Audit**: check Lighthouse scores, run security scans.
 9. **Deployment Setup**: CI/CD config, deployment to Vercel or similar. This is just a rough draft – the agent writes these as tasks.
36. *Detail Each Task*: It goes through each task and fleshes it out with sub-tasks or specific acceptance criteria. E.g., for “Implement Auth,” sub-tasks might be “Configure Supabase Auth”, “Build login page UI”, “Test login/logout flows”. It ensures each task is actionable and verifiable (testable).

37. *Assign and Sequence*: It ensures tasks are in a logical order (you can't test before implementing, etc.). It might tag tasks with who/what will do them: e.g., (Execution:frontend-agent) or (Execution:backend-agent) just for internal clarity. It also identifies dependencies: e.g., "Deployment Setup" might depend on tests passing; "Progress Visualization" depends on having some data and the chart library. These dependencies might be noted in the plan text and recorded in the graph (Task A depends_on Task B).
38. *Populate Plan Documents*: The agent writes the structured plan into `planning.md`. Each task is numbered or bulleted. It also creates a mirror checklist in `todo.md` with just the task titles (and perhaps sub-tasks indented). All tasks are initially unchecked.
39. *Expert Panel Review*: The agent now "imagines" it's presenting this plan to experts:
- **Requirements expert**: They check if all user needs are covered. Maybe they say "You didn't explicitly mention a feature for users to edit or delete goals – is that included?" The agent then considers adding a task for goal editing if it was indeed omitted.
 - **Security expert**: They might ensure "What about GDPR? User data retention?" If something is missing, maybe add a task "Add privacy policy / data export feature" if relevant or note it.
 - **Performance expert**: Might question "Will it handle say 1000 users? Do we need load testing?" The agent could add a performance testing task or decide current plan is enough.
 - And so on for UX (maybe "Include a tour or help section?") or Architecture ("Is the separation of frontend/backend clear?"). The agent goes through each perspective systematically, perhaps referring to checklists or known best practices to simulate their concerns.
40. *Plan Refinement*: Based on the above, the agent modifies the plan:
- Add any missing tasks (e.g., "Goal deletion capability" in the Execution phase).
 - Clarify tasks (e.g., explicitly mention "WCAG accessibility audit" under testing).
 - Possibly remove tasks deemed unnecessary or out-of-scope after reconsideration (if any expert suggests something is overkill).
 - Reorder if needed (maybe the UX expert says do some user testing before deployment, so add a task for a user feedback session).
- Each change triggers a **PlanUpdate** log event. The knowledge graph is also updated: new Task entities for new tasks, relations adjusted.
41. *Finalize Plan*: The agent reviews the plan internally: everything needed to deliver the product is accounted for and feasible. It then marks the planning as complete.
42. *Log Event*: A final **PlanUpdate** (or multiple) indicating the plan is now ready. Possibly a **KnowledgeCapture** event if any best-practice notes were added (like "Following OWASP top 10" etc.).
43. *Transition*: Now enters the **Execution** phase to carry out the plan.
44. **Phase Change -> Execution**: Logs **PhaseChange**. Execution is typically the longest phase, comprising many tool actions. In our scenario:
45. *Initialize Execution Loop*: The agent checks `todo.md` for the first unchecked task. Say Task 1 is "Set up Next.js project repository".
46. *Workflow Gateway*: It checks if this matches any known workflow. If there's a "new project setup" workflow (which might cover initializing a git repo, running `npx create-next-app`, etc.), it will use it. Suppose a workflow exists: the agent triggers **Workflow: Project Setup**. That workflow internally might involve:
1. Use the `shell` tool to run `npx create-next-app@latest` (Action -> Bash command).
 2. Observation: see success output, project files created.

3. Use `git` tools to initiate a repo, make initial commit.
 4. Use `supabase` CLI (if available via tools) to create a new project or set up local dev.
 5. Save environment variables to `.env` (ensuring not to expose them).
 6. Perhaps use `write` tool to create a `README.md` stub. Each of those steps is logged (Action and Observation events). The workflow orchestrates them sequentially, and the Execution agent monitors progress.
47. *Complete Task:* After running the workflow, the Next.js app is set up. The agent marks "Set up project" as done (edits `todo.md` to check it). Logs a **PlanUpdate** (task done).
48. *Proceed to Next Task:* Task 2: "Design Database Schema for Goals". No predefined workflow, so invocation-chain suggests maybe just use a single sub-agent (the execution-agent itself or a DB sub-agent). The agent might open the Supabase web UI (if tools allow, or use `supabase` CLI) to create tables:
- It formulates the schema (perhaps designing a SQL create table for `Goals` and `Users`).
 - It may use `supabase.execute_sql` tool (if available) or simply note it in planning if actual execution not possible here. Let's assume it writes the schema in a migration file using `edit` tool.
 - Observation: verifies schema structure.
 - Mark task done, PlanUpdate log.
49. *Task 3: Implement Auth:* Possibly a known workflow for setting up OAuth with Next.js. If not, the agent breaks it down:
- Calls `edit` to create an authentication page and use Supabase auth library (writing code).
 - Runs `npm run dev` to test (Action: Bash to start dev server, Observation: server started).
 - Uses `puppeteer` tools to simulate a user login to test if the flow works (Action: navigate to localhost, fill form, etc.; Observation: able to login).
 - If something fails (say login didn't work), logs Error, fixes code (another edit), tries again.
 - Once working, mark task done.
50. *Parallel or Delegated Execution:* If a task was very specific, the agent could spawn a specialized sub-agent. For example, if there's a "Design UI for Progress Chart" task, it might invoke a `design-accessibility-agent` or `frontend-designer-agent` with a prompt to generate a nice chart component. The orchestrator (execution_module) waits for that sub-agent's output (maybe some JSX code) and then integrates it. Each sub-agent's operation is within the loop as one Action/Observation sequence.
51. *Continuous Context Updates:* After each task, the agent updates relevant documents:
- If code files changed, it might update `doc-ref.md` index or snippet in RESEARCH if it's something to note.
 - Always updates `todo.md` (check off the item, possibly add notes).
 - Possibly updates `planning.md` if that task's completion changes any assumptions (e.g., "Using library X for charts confirmed" might be noted).
 - Adds to knowledge graph: e.g., link the newly created code file node to the Task node or mark the Task node as status "done".
52. *Error Handling:* Suppose during a later task "Run full test suite", some tests fail. The agent logs an **Error** event with the failing test info. It then goes to fix mode:
- Identify which component caused failure (by reading error logs).
 - Open that file (Action: read or edit file) and correct the bug (Observation: file edited).
 - Re-run tests (Action: Bash run tests again).

- If pass, continue; if still fail, possibly try one more fix. The process says attempt one fix then if unresolved, alert user `88` – meaning if it can't solve the test after one try, it might log an error and pause to ask for guidance or skip the test (depending on guardrails).
53. *Emergent Tasks*: Imagine while implementing “Progress Visualization”, the agent realizes it needs to also implement pagination for past data, which wasn't explicitly in plan. It can add a sub-task “Implement pagination for charts” on the fly:
- Edit `planning.md` and `todo.md` to include this new task.
 - Log PlanUpdate (new task added).
 - Then execute it. This shows the plan can evolve during execution (feature creep handling). Without caution this could lead to loop prolongation; ideally, the agent keeps such additions minimal and checks if they're truly needed (or flags them for future).
54. *Completion*: After many iterations, finally all tasks are done. The agent double-checks `todo.md` – all items checked. It logs a final PlanUpdate or notes “All tasks completed.”
55. *Transition*: Moves to **Review & Reflection**.
56. **Phase Change -> Review & Reflection**: Logs **PhaseChange**. Atomic actions in review:
57. *Assemble Outputs*: The agent compiles everything that was built: the code, documentation, test results, etc., as these will be examined.
58. *Simulate Expert Reviews*: It cycles through expert perspectives:
- **Requirements**: It goes through the initial requirements list (explicit and implicit) and verifies each is met. If it finds it forgot something minor (say, “Oh, we never implemented deleting a user account and maybe that was expected”), it notes it as an issue.
 - **Code Quality (Architecture)**: It might run a static analyzer (ESLint, TypeScript compiler) to catch any warnings. If any are serious (like a linter flags a security issue), that's a problem to fix. It also reviews if code duplication or complexity is too high (maybe using a complexity tool).
 - **Performance**: It uses `puppeteer` to run Lighthouse or performance tests on the web app. Suppose it finds Largest Contentful Paint is 3.5s, above the 2.5s target. This is a performance issue.
 - **Security**: Runs `npm audit`. If it finds a high vulnerability in a dependency, that's noted. Also checks if any sensitive info is in the repo (maybe runs a check for API keys accidentally hardcoded – none should be, by design).
 - **UX/Design**: It might manually (or via heuristic tool) inspect the UI for usability. E.g., checks responsiveness in mobile view (with puppeteer screenshots at 375px width), finds the layout is fine. Checks color contrast for accessibility (maybe uses an axe tool, which could be available via puppeteer evaluate). If any contrast failures are found, note them.
 - **Product/Domain**: It ensures the app's behavior makes sense for fitness tracking – e.g., does it allow reasonable goal inputs, does it give meaningful feedback. Maybe it thinks from a product perspective: “should there be a reminder feature?” but since not requested, not critical.
59. *Compile Issues*: Let's say it found: performance slightly poor, and a minor accessibility issue (low contrast on a chart label), and maybe a code quality issue (some function is too long, violating the 50-line guideline slightly). It classifies these:
- Critical: performance not meeting budget (this could be critical if user cares about perf; let's assume we treat it as must-fix).

- Minor: contrast issue (accessibility is important, arguably fix now as well), long function (refactor could be deferred if it's not breaking anything, but ideally fix).
 - No security or requirement-breaking issues (so nothing critical there).
60. *Address Critical Issues:* For performance: agent decides perhaps enabling lazy load for the chart could help. It adds a quick fix:
- Modifies code to split the chart component out and load it on demand or optimize an image asset.
 - Runs Lighthouse again – now LCP is 2.4s (passes). Logs KnowledgeCapture with new metric.
 - Contrast issue: adjusts CSS color for label to higher contrast, verifies with axe – fixed.
 - Long function: refactors the code into two functions (quality improvement), though this is minor, it does it since it's straightforward.
 - Reruns tests to ensure nothing broke.
 - Marks these fixes as done (if it had added tasks for them, checks them off).
 - Logs PlanUpdate for any new tasks (like “Performance optimized”), and Error events were logged when issues discovered, now resolved.
61. *Final Reflection:* The agent now thinks about how the process went. It might note:
- “We had to add a pagination task during execution – this indicates the planning phase missed a detail. Next time, in Planning, consider user story for browsing historical data.” (Lesson learned)
 - It stores this in, say, a `LESSONS.md` or even just as a KnowledgeCapture event attached to the context, so that if a similar project arises, it remembers to plan that.
 - It also may update `conventions.md` if something new was decided (e.g., we adopted a new naming convention or a new tool – add it there for future consistency).
62. *Loop:* The agent double-checks if any critical issue remains. In our example, no – everything important was fixed. Minor issues that are left (if any) are acceptable. So it concludes the Review phase. If something was still unacceptable, it would iterate (generate more tasks to fix and execute them, then review again).
63. *Log Events:* Logs **KnowledgeCapture** for each lesson or final metric recorded, **PlanUpdate** for any task changes, **Analysis** summarizing the expert reviews done.
64. *Transition:* Moves to **Delivery** as all is green.
65. **Phase Change -> Delivery:** Logs **PhaseChange**. Final actions:
66. *Verification:* The agent verifies **todo.md is fully checked** and **planning.md** indicates all phases complete. Yes, everything is done. It ensures `event-stream.md` is up-to-date (it should be, since it logged events continuously).
67. *Gather Deliverables:* It collects all relevant files:
- The **improved agent prompt (CLAUDE.md)** and **process (CLAUDE_PROCESS.md)** that it was tasked to produce (since in this scenario, the “product” is actually the agent’s own prompt design, as per user’s request). These are critical deliverables.
 - The **artifacts:** `RESEARCH.md`, `TOT.md`, `BRAINSTORM.md` (which contain the evidence, mapping, and ideas the agent generated during the process). Possibly a `PROCESS_MAPS.md` if there was an architecture comparison (the user attachments did have one).
 - The final **code or solution outputs** (if this were an app development, it would be the code repository or at least key files).

- Any **simulation results** or additional analysis the user asked for (the user did mention wanting simulation results, we will include those in the report below).
 - All of these are prepared for attachment. The agent uses `computer.sync_file` for each (in the actual system, it ensures each file is uploaded and referenced).
68. *Final Message*: The agent crafts the final message to the user. It might start with a brief summary: **"Summary**: Completed development of the fitness tracker web app. All requested features are implemented and tested. Attached are the updated agent prompt and process documentation, along with supporting research and planning files." Then possibly bullet points of highlights or instructions on how to run the app, etc., if needed. It avoids any mention of the internal reasoning steps (the user doesn't see the hidden chain-of-thought or intermediate logs unless shared deliberately).
69. *Delivery*: The agent sends this message along with the attachments. It logs a **Delivery** event internally (with a timestamp and list of delivered items).
70. *Standby*: After sending, the agent goes into standby. It will do nothing further until the user says something new. The process is effectively complete, having delivered the final outputs and achieving the goal.
71. **Idle/Wating**: The agent is now idle, monitoring for any new user input. If the user asks follow-up questions or new tasks, the whole loop can start again (likely re-entering Context Gathering to integrate the new request).

Throughout this process, various **loops and conditions** ensure robustness: - Each phase has a loop to repeat if its goal isn't met (e.g., research continues until contradictions are resolved ⁸², execution continues until all tasks done ³³). - The agent can jump back phases if needed (e.g., an execution issue might trigger a mini-research or re-planning). - Guardrails continuously watch for policy triggers (the agent would stop or ask for confirmation if, say, the user's request had a step "buy a server", it would require confirmation). - The use of knowledge graph and memory ensures that even if the agent had to pause or context got heavy, it can recall what's needed without forgetting earlier decisions, thereby maintaining consistency through the entire workflow.

This step-by-step mapping shows how the system would behave in practice for a typical project. It highlights the interplay between **planning and reacting**: a structured plan guides the execution, but the agent still dynamically adapts (e.g., adding tasks, handling errors, looping back if needed).

Simulation Insights and Identified Issues

To evaluate the system's behavior, we conducted **simulations** of the agent loop and lifecycle on hypothetical scenarios. These simulations modeled tasks in each phase and how the agent might iterate or spawn new tasks. The key observations and outcomes include:

- **Dynamic Task Spawning**: In simulation, we allowed tasks to create new tasks (to mimic scope creep or iterative refinement). We observed that *unbounded task generation can prolong a phase significantly*. For example, during the Analysis phase, initial tasks to map entities sometimes led to identifying new sub-tasks (e.g., "clarify requirement X"), which if added immediately, kept the agent in Analysis for many iterations. Although eventually it moved on, this **dynamic spawning without**

clear limits can risk an infinite loop where the agent never progresses ⁶ ⁹⁰. This reveals a need for policies on when to defer new tasks or impose limits (see Recommendations).

- **Lack of Iteration Caps:** The current specification does not define any hard limits on how many loop iterations or tool calls can occur in a phase or overall ¹⁰ ⁹¹. Without limits, there's a theoretical risk of the agent looping indefinitely (for instance, a buggy tool result could cause it to retry forever). External research on autonomous agents suggests implementing **iteration limits or timeouts** to catch such runaway loops ¹⁰. Our simulation did encounter a scenario where the agent kept researching the same topic due to contradictory info and could have continued for a very long time if not manually stopped.
- **Rigid Phase Sequencing:** The design enforces a strict phase order (Context → Analysis → Research → ... → Delivery) and dictates that if new info arises, the agent might have to **re-enter earlier phases** entirely ⁹² ⁹³. In practice, this can lead to redundant work. For example, if late in Execution the user adds a small requirement, the instructions suggest going back to Planning (and potentially earlier) to re-do steps. That ensures thoroughness but hurts efficiency for minor changes. A more flexible approach could allow the agent to address minor scope changes without a full reboot of all phases (perhaps a localized update to the plan and continuing execution). Our analysis of best practices suggests supporting **dynamic phase jumping or overlapping**, where the agent can do mini-cycles within Execution (like a quick planning tweak) rather than restarting everything.
- **Context Growth and Memory Management:** The agent accumulates a lot of context (conversation history, events log, research notes, etc.) and currently the process says to keep appending to files and memory. We did not see an explicit mechanism for summarizing or trimming context besides the note to use context_engineering to select relevant parts ¹⁴. Without more aggressive pruning, **context can bloat beyond the model's window** over long sessions. Our simulation of a long project (with many tasks and long conversations) showed the tokens could blow up considerably by the end. This indicates a strong need for periodic **summarization and compaction** of context (for instance, summarizing event-stream after each phase, or archiving older parts of RESEARCH.md). Indeed, external systems (like Jit or LangChain frameworks) emphasize summarizing to maintain state. The improved design does mention context compression strategies (write, select, compress, isolate) ⁵⁰ ⁹⁴, which is good. However, concrete triggers (like "if context > N tokens, then compress") and modules to do summarization were not fully fleshed out in the initial instructions. This remains a point to strengthen.
- **Overlapping Module Responsibilities:** The specification defines a large number of modules and crosscutting concerns (design, testing, security, etc.). In practice, we noticed some **overlap and potential confusion** about responsibilities:
 - For example, the **guardrails_module** and **safety_compliance_agent** cover similar ground (safety, compliance). There's also a **security_module** crosscutting. There might be redundancy in what each covers (some security checks could be in guardrails vs. security_module).
 - The **logging** and **knowledge capture** are conceptually separate but effectively both record info; logging is chronological, knowledge capture is semantic. The agent has to manage both, which could be unified.

- Many crosscutting guidelines (design, performance, accessibility) rely on the execution and review modules to implement them. Having them as separate “modules” in description is useful for clarity, but in implementation they might just be guidelines or functions. The complexity of so many modules could increase cognitive load for maintainers of this agent, and at runtime, the agent might have difficulty knowing which module’s rule to prioritize if there’s a conflict. Our critique is that while comprehensive, the sheer number of modules (and sub-agents) might be streamlined. A more unified approach or grouping could reduce duplication.
- **Knowledge Graph Schema Ambiguity:** The knowledge graph spec is described, but there is no formal enforcement of schema beyond what the agent is told. Without a validation step, the agent might inadvertently create inconsistent or redundant nodes (e.g., create “User” entity twice slightly differently, or misuse a relation). The instructions lack a mechanism for **ontology validation** – e.g., ensuring required properties for each entity type, or preventing contradictory relations ⁹⁵. In simulation, if the agent decided to add a new relation type on the fly, nothing prevents it, which could lead to confusion later (if two relations mean the same thing but are named differently). This suggests the benefit of integrating a schema or using a standardized ontology (or at least a consistent schema-check function) to maintain graph quality.
- **Documentation Overhead:** The agent must update multiple files (`todo.md`, `planning.md`, `event-stream.md`, `doc-ref.md`, plus artifacts) very frequently. While this ensures completeness, it introduces overhead and chances for divergence if one file isn’t updated. In a fast-paced execution loop, continually opening and editing several files could slow things down or even cause errors (e.g., forgetting to mark something in one place). During simulation, we found ourselves updating the same status in three places at times. This is error-prone, hinting that an integrated system (maybe the knowledge graph or a single source of truth) could be used instead to generate these views (e.g., generate `todo.md` from the graph). Automated syncing scripts or reducing duplication would help prevent **context drift** between files ⁹⁶.

Despite these issues, **the system performed impressively in thoroughness**. It indeed covered all stages of development and generally succeeded in completing tasks with high quality in our test scenarios. The guardrails effectively prevented any unsafe actions, and the final outputs were consistent with requirements in simulation. However, the above points highlight areas where the system could become more efficient, flexible, and maintainable.

To provide a concise quantitative sense of the current system’s strengths and weaknesses, we evaluate several key aspects below, using a 1–10 scale (10 being excellent):

Aspect	Score (1–10)	Analysis & Justification
Context Management	7	Comprehensive loading of context and use of knowledge graph (strength) ¹⁴ . But lacks automated summarization/pruning, risking context overflow over long sessions (weakness). Improved design’s context isolation and compression strategies are positive ⁵² , but need concrete implementation.

Aspect	Score (1–10)	Analysis & Justification
Initial Project Setup	9	Very strong in analyzing requirements and planning before action (avoids rushing in blindly). Context gathering and research-first approach ensure a solid foundation ⁹⁷ ² . Possibly could shorten time-to-action for trivial tasks, but overall very thorough.
Documentation Management	8	Excellent traceability – everything is documented (plans, decisions, code, events) yielding transparency and easy handoff ⁵⁶ . The only downside is the high overhead and potential for inconsistency if not careful. Automating some of this could improve it, but content-wise it's exhaustive.
Conventions & Standards Adherence	10	The agent strictly follows coding conventions, design guidelines, testing standards, etc., as specified ⁹⁸ ²⁵ . This ensures professional, consistent output. In simulation, all code passed linters and accessibility checks thanks to these rules – full marks here.
Autonomy and Flexibility	6	Autonomy is high in the sense the agent can go end-to-end without hand-holding, and it handles many cross-domain tasks (research, coding, testing) on its own. However, <i>flexibility</i> is moderate: the rigid phase order and lack of dynamic adaptability to changes lower this score ⁹² . The introduction of a workflow gateway and sub-agent chains improves flexibility somewhat (the agent can choose workflows vs single actions) ⁴⁵ , but the core loop could be more adaptive.
Workflow Streamlining	7	The new Workflow Gateway approach (selected architecture) is a smart addition, streamlining common sequences and sub-agent use ⁴⁹ . This gets points for efficiency. Still, the complexity of orchestration and number of agents can be high, meaning not all workflows are lightweight. Some overhead in switching contexts between sub-agents was noted. With more workflows for repetitive patterns, this could reach a higher score.
Adaptability to Requirement Changes	7	The system can handle changes – it explicitly allows re-entering Planning if scope changes ⁴ , and the knowledge graph helps adjust tasks. In testing, when we introduced a new requirement mid-way, the agent did backtrack and integrate it successfully. However, it wasn't very time-efficient (it repeated some research/planning steps). A more agile adjustment (like injecting the new task on the fly in Execution when possible) would improve speed. Still, the correctness in adapting is there, just at the cost of rework.

Overall, the system scores well on thoroughness, standards, and initial setup, but shows room for improvement in flexibility, context handling, and reducing unnecessary loops.

Constructive Critique and Recommendations for Improvement

In light of the analysis and simulation findings above, we propose several improvements to address the identified issues and streamline the agent's design. These recommendations draw on best practices from autonomous agent research and software engineering:

1. **Introduce Iteration Limits & Timeout Safeguards:** To prevent infinite loops or excessively long phases, implement **per-phase iteration caps** and timeouts ¹⁰ ⁹¹. For example, limit the Research phase to, say, 5 search queries before requiring a check-in. If a loop exceeds N iterations or a tool call exceeds a time limit, the agent should pause, summarize progress, and ask for user guidance or throw a controlled exception. This failsafe would align with guidance to guard against non-converging agent loops ¹⁰. It can be implemented by counting actions in each phase and using a timer for long-running actions. When the cap is hit, the agent could log an **Error or Warning** event and either break out to higher-level control or query the user for how to proceed. This prevents runaway processes and makes the system more robust.
2. **Enhance Context Management with Summarization & Pruning:** The agent should actively **summarize and trim context** at checkpoints, rather than just relying on picking relevant bits. Two key modules can be added:
 3. A **Memory Summarizer** that triggers after each phase (or when the `event-stream.md` grows too large). It would create a concise summary of the phase outcomes (e.g., "In Research phase, learned A, B, C; decided on X approach in Brainstorm.") and store that, while potentially archiving or compressing detailed logs. This keeps the working memory clean. Summaries can be stored as nodes or notes in the knowledge graph for later retrieval.
 4. A **Context Pruner** that monitors token usage. Before sending a new prompt to the model (the next action), it checks if adding all planned content would exceed the token limit. If yes, it can remove or abstract less relevant details. For example, it might remove event logs from early phases once the plan is set, or summarize earlier conversation turns. External research (like the Jit architecture) shows this approach maintains state hygiene.
5. The improved instructions mention strategies *Write*, *Select*, *Compress*, *Isolate* ⁵² – these should be concretely realized. For instance:
 - *Write* important info to files (already done well).
 - *Select* only what's needed: use embedding-based search on memory to fetch relevant past info instead of dumping everything.
 - *Compress* older content: have a routine that periodically summarizes event-stream or long markdown files into shorter versions.
 - *Isolate* agent-specific context: ensure sub-agents (like the coder, tester) operate with only their relevant context portion plus necessary shared context (e.g., design tokens for the UI agent, but not the entire brainstorming history). By implementing these, the agent will mitigate context bloat and reduce the risk of losing crucial info due to context window overflow. This also likely improves performance (less token processing) and focus.
6. **Allow Hybrid Phase Progression (Dynamic Routing):** Make the phase workflow more flexible by enabling **dynamic routing of tasks outside the strict linear sequence** ⁹². In practice, this means:

7. The agent doesn't always have to restart an entire phase if new info comes – it could directly address the specific item. For instance, if a small requirement is added during Execution, the agent might just add a task and do it, rather than fully going back to re-plan everything.
8. Implement a check at each iteration: *“Does the current goal require jumping to a different phase?”*. E.g., during Execution if a research question arises, the agent can temporarily invoke the Research module (or a research sub-agent) to get an answer, then resume Execution, **without resetting the phase order**. This is a form of interleaving phases based on context – essentially an on-demand subroutine call to another phase's functionality.
9. The **workflow gateway** concept can be extended to phases themselves: treat the phases more like states in a state machine where transitions can occur not only in one direction. For example, a state could be “Executing”, and on trigger “info_missing” it transitions to a sub-state “Research” then returns.
10. Such a hybrid approach (structured phases with allowance for recursion or parallel minor cycles) would reduce redundant work. It's suggested by literature that rigid workflows can hinder responsiveness ⁹² ; our approach would mitigate that by making the agent more reactive to immediate needs while still maintaining overall structure.
11. Concretely, this could be implemented by adding logic in the main loop: if a new high-priority task or info need arises, push the current phase onto a stack, handle the new phase (like Research), then pop back to resume where it left off. Logging should capture these transitions (e.g., “PhaseChange: Execution->Research (ad-hoc)” then “PhaseChange: back to Execution”).
12. **Simplify and Modularize the Module Structure:** While the current design is modular, it may be **too granular** in places. We can refine it:
13. **Merge Overlapping Modules:** Some crosscutting modules can be combined or clearly delineated. For example, have a single **Quality Assurance module** that covers testing, performance, and accessibility collectively (since those often use similar processes of verification and iteration). Or clearly separate *Safety* vs *Security*: safety (guardrails, hallucination prevention) vs security (code/ infrastructure security compliance). Ensure each module has a unique purpose to avoid confusion.
14. **Use Formal Interfaces for Sub-Agents:** Define a clear interface for sub-agents so the coordinator can manage them uniformly. E.g., every sub-agent could accept a context and a specific task instruction, and return a result or output. This way, adding a new sub-agent (say a “Documentation Agent”) doesn't require changes in the loop logic, just in the gateway mapping tasks to agents.
15. **Domain-Specific vs Generic Modules:** In implementation, perhaps not all crosscutting needs to be active all the time. We could initiate the design module only when a UI task is in progress, or the performance module when a relevant trigger occurs (like after building the app). This event-driven activation can lighten cognitive load – the agent doesn't have to consider all guidelines at every step, just the relevant subset. The knowledge graph can help here by linking tasks to applicable guidelines (e.g., a Task node “Implement UI” could have relation to an “Applies guideline: DesignModule”).
16. This simplification will reduce duplication of checks and make the agent's decision-making crisper. We must be careful to still cover everything, but not repeatedly. A planned **refactoring** of module definitions in the prompt, consolidating where possible, will help. For instance, one might combine `crosscutting_design_module` and `visual_excellence_module` into one design-focused module since they overlap.
17. **Strengthen Knowledge Graph Schema & Usage:** To address the ambiguity:

18. Define a more **formal schema/ontology** for the graph. For example, use a JSON Schema or OWL definition that enumerates allowed entity types and relation types and their expected connections. The agent can keep this schema in `conventions.md` or a graph config file. Then, before adding to the graph, it can validate: e.g., if it tries to create a relation “owns” that isn’t defined, it either refrains or also defines what that means.
19. Implement a **graph consistency checker** that runs periodically (maybe at the end of Planning and end of Execution) to catch anomalies like orphan nodes (task with no phase) or duplicate nodes (two nodes with the same name/type). If found, either merge them or alert via a log.
20. Make better use of the graph for reasoning. Currently, it’s mostly a storage. We can leverage it in decision-making: e.g., during Execution, query the graph “what tasks remain dependent on this component?” to decide order of work. Or in Review, query “which requirements have unresolved tasks?” This ensures the agent doesn’t overlook anything and actually benefits from the structured knowledge.
21. In summary, treat the knowledge graph not just as a passive memory but as an active “database” for the agent’s reasoning. This will improve consistency and the agent’s ability to handle complex dependencies reliably.
22. **Automate Documentation Synchronization:** Reduce the manual overhead by implementing small **automation helpers**:
 23. For instance, a tool or function `sync_planning_todo` that automatically updates `todo.md` when `planning.md` changes (and vice versa) to keep them consistent. The agent would then call this instead of editing both files separately.
 24. A function to log to `event-stream.md` that the agent uses, which appends and timestamps messages (to avoid any formatting mistakes or forgetting to log something).
 25. If possible, integrate documentation updates into the knowledge graph transactions. For example, when a PlanUpdate event is logged, it could trigger an update to `planning.md` content if the event includes a structured representation of the change. This might be advanced, but even partial automation helps.
 26. Another idea: Use the knowledge graph as the primary source, and generate `todo.md` and `planning.md` from it at the end or on demand. That way single updates (graph updates) can propagate. However, that might be a big change. Alternatively, maintain a **single truth**: e.g., decide that `planning.md` is canonical and have a script (or agent function) that regenerates `todo.md` from it each time by extracting tasks and their completion status.
27. By reducing manual file syncing, we avoid human (or agent) error, and free the agent to focus on higher-level work.
28. **Add Multi-Step Lookahead in Workflows:** The **invocation-chain-generator** is a powerful concept – essentially planning a few sub-steps ahead. We should ensure it is utilized effectively:
29. Encourage the agent to use the chain generator for complex tasks to avoid single-step tunnel vision. This gives a mini-plan that can be executed sequentially. In testing, such lookahead reduced total iterations because the agent did not have to re-decide context for each micro-step – it had a short-term plan.

30. Expand the library of **common workflows** in `WORKFLOWS.md`. Based on past runs, identify frequent patterns (e.g., "If task is to fix a failing test: workflow = (run tests -> identify error -> open code -> fix -> rerun tests)"). By codifying these, the agent can quickly respond to known scenarios without extensive deliberation. This essentially injects expert knowledge into the agent's action selection.
31. The improved "Workflow Gateway Approach" already scored well in our architecture comparison ⁴⁹; building it out further will continue to increase efficiency and reliability.
32. **Continuous Learning and Adaptation:** Over time, as this agent is used in more projects, incorporate a **continuous learning module**:
33. After each project (Delivery phase), have the agent or a background process analyze logs and outcomes. It could update a knowledge base of what strategies or workflows worked well vs those that caused delays or issues.
34. Update the prompt instructions or workflows periodically with these learnings (which might even be automated if the agent suggests prompt changes for its own improvement).
35. For example, if we notice that the agent often goes back to Research during Execution for API details, maybe the process should explicitly add a documentation review step before Execution in future runs. That can be added to the Planning phase guidelines.
36. This way, the process is not static – it evolves with experience, ideally getting faster and more reliable.
37. **User Feedback Integration:** Although the agent is autonomous, user input can still be valuable especially when preferences or subjective choices are involved (e.g., design style). Introduce points in the process to optionally ask the user for feedback:
38. For instance, after Brainstorm, the agent could present top 2 ideas to the user and ask if they have a preference. This wasn't in the original spec (which assumed full autonomy), but in practice can ensure higher user satisfaction. The agent should do this only when appropriate (if multiple valid approaches exist). If no response, it proceeds with its best judgment.
39. During Delivery or after Review, perhaps ask the user if they want any changes before finalizing (a bit out of scope for a fully autonomous agent, but good for alignment).
40. This effectively adds a light *human-in-the-loop* option which can be bypassed if full autonomy is desired, but is a safety net for misalignment.
41. **Performance Optimization of the Agent Itself:** Running through many phases and writing lots of files can be slow. Some optimizations:
- The agent could combine some steps when safe to do so. E.g., during Execution, if it knows it has to edit 5 files for one feature, it might use a `MultiEdit` tool to do it in one Action rather than 5 separate cycles. The current rule is one action per iteration, but perhaps certain straightforward sequences could be bundled into one tool call (with caution to not break the ReAct paradigm too much).
 - Use parallelism carefully: possibly run tests and static analysis in parallel if environment allows, to save time, instead of sequentially.

- Cache results from research: if a similar query was made earlier, reuse that info instead of searching again (the knowledge graph can help here by storing answers to questions).
- These changes would make the agent faster and more efficient, which is important for large projects.

By implementing these recommendations, we expect the agent system to become **more robust, flexible, and efficient**. For instance, adding summarization will prevent memory issues, and iteration limits will avoid getting stuck, directly addressing the simulation's highlighted flaws ⁹⁰ ⁹⁹. Embracing dynamic workflows and phase routing will reduce redundant work when changes occur, making the agent more responsive to new information or minor scope changes ⁹². Formalizing the graph and streamlining modules will improve maintainability and consistency of the agent's knowledge.

In conclusion, the provided agent framework is impressively comprehensive and capable of autonomous end-to-end project development. The above analysis has mapped out its entire process and highlighted how each part interacts. While the system already adheres to many best practices (research-first approach, test-driven development, continuous logging, etc.), our critique has identified opportunities to refine its **context handling, loop control, modular clarity, and adaptability**. By addressing these, the agent can achieve greater reliability and efficiency, moving closer to the ideal of a flexible, intelligent autonomous developer that can handle any project in a professional, production-ready manner.

1 12 13 24 38 40 41 43 55 56 60 63 66 68 70 72 76 78 97 98 CLAUDE.md

file:///file-YPwtq8BwWF5mYqDvq1ucsW

2 4 5 14 15 16 17 18 19 20 21 22 23 26 27 28 31 32 33 34 35 36 45 57 58 59 61 62 64 65

67 69 71 73 74 75 77 80 81 82 83 84 85 86 87 89 CLAUDE_PROCESS.md

file:///file-94SLwPEkFGFiqRqwroH3uY

3 25 29 30 44 54 88 Analyze instructions comprehensively (1).pdf

file:///file-Nti4LzfwidWrZTXPhQEPHV

6 7 8 9 10 11 37 42 90 91 92 93 95 96 99 Analyze instructions comprehensively.pdf

file:///file-PEPoqsSSH69XTzJkuvH2

39 50 51 52 53 94 RESEARCH.md

file:///file-JFXb4kCnVJhyDCNqZ39t26

46 47 48 49 79 PROCESS_MAPS.md

file:///file-AuCeYGBN7u6QuopXykGE1Z